

Assignment 3 : Property Price and Type Prediction

1. Challenges

Challenges in House Price Prediction (Regression)

Predicting house prices is complex due to several factors:

- **High Variability in Prices:** Property prices depend on numerous dynamic features like number of beds, location, property type, region, size, and market conditions.
- **Outliers and Skewness:** Real estate markets often have outliers (e.g., certain properties like Block of houses have high number of rooms whereas Studios and vacant land have 1 and 0 rooms respectively) that skew the data.
- **Temporal Volatility:** Prices fluctuate based on macroeconomic and seasonal trends based on features Property Inflation Index and Cash rate. There is also a huge boom post to pandemic era in 2021.
- **Multicollinearity:** Many features are correlated, complicating model interpretation. Features like `time_to_cbd_driving_town_hall_st` and `km_from_cbd` are highly correlated.
- **Missing values :** Even though the dataframe tells it doesn't contain null values there are a lot of zero values ('0' and 0) in the columns. I set a threshold to drop the values if it has more than 50% missing data.

Challenges in House Type Prediction (Classification)

- **Imbalanced Classes:** Some property types (e.g., House, Townhouse) are much more common than others.
- **Overlapping Features:** Apartments and townhouses may have similar characteristics, making it difficult to distinguish them. For Example, the Apartment prices in the city are closer to the house prices that are about 50 km from CBD.
- **Multicollinearity:** Many features are correlated, complicating model interpretation. For predicting the house type, there are a lot of irrelevant features.

Strategies to Address These Challenges

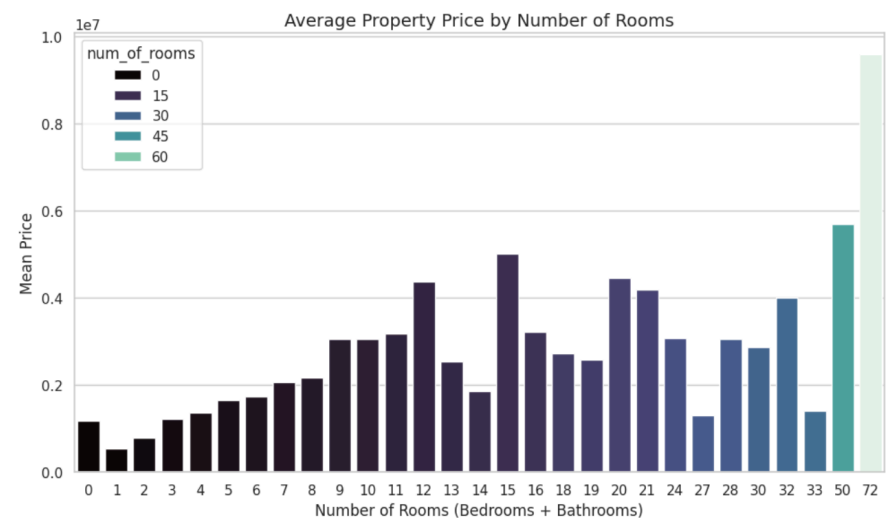
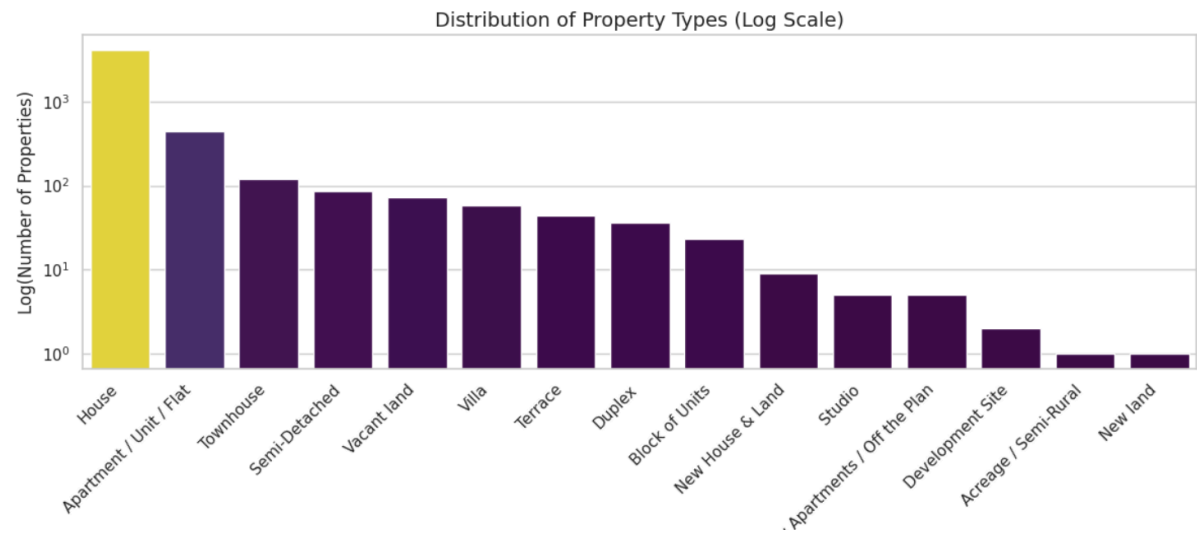
- **Feature Engineering:** I have derived new features like `num_of_rooms`, `is_sold_in_2021` as the year 2021 saw a sudden rise in the property prices, `inverse_cbd_distance`, `is_cbd`. I have mapped the regions based on the mean prices in the training set.
- **Random Forests Classifier:**
 - House types may not follow a linear pattern (e.g., suburb + price + room count could relate to multiple types). RF captures complex, nonlinear patterns.

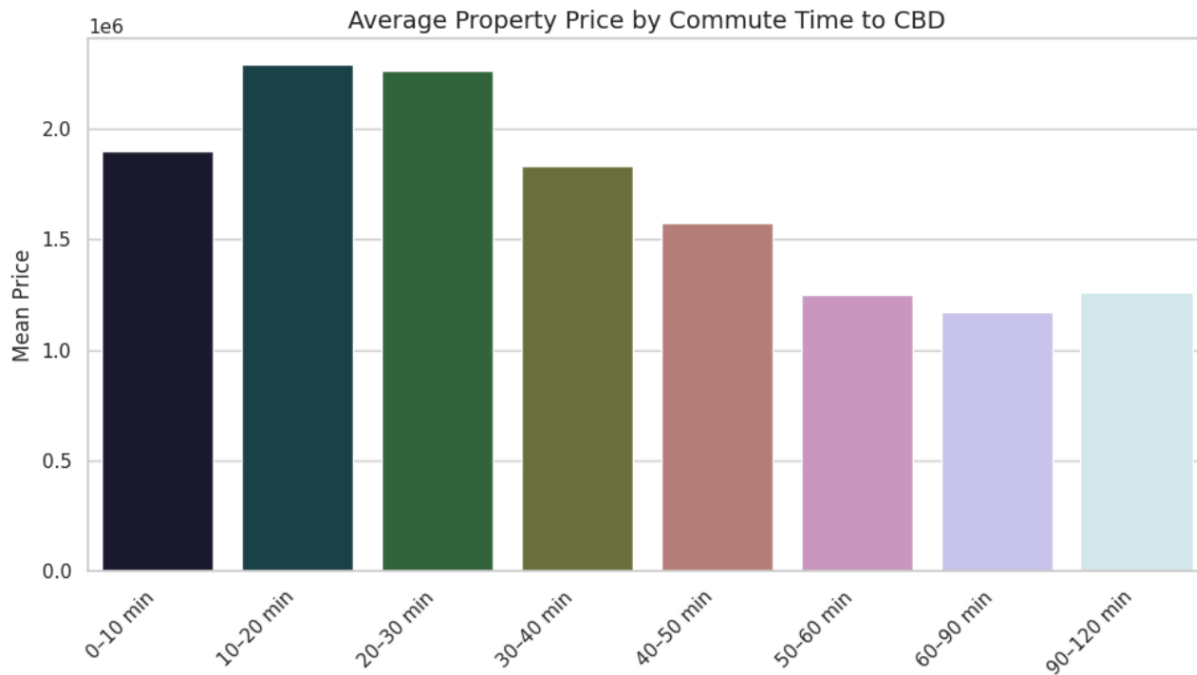
- Real-world housing data has noisy entries (weirdly low/high prices, missing values filled with means, etc.). RF is robust against this.
 - The Data is imbalanced (House: 4000+ entries vs. some with <10). While not perfect, RF has the `class_weight='balanced'` option to counteract this.
- **HGBRegressor**
 - Housing prices don't increase linearly with features like `num_bed`, `commute_time`, or `property_size`. HGBR captures complex patterns automatically.
 - HGBR is optimized for speed and memory usage. If your dataset has thousands of entries and dozens of features, HGBR performs better than older gradient boosting frameworks like vanilla `GradientBoostingRegressor`.
 - You don't need to scale features (no need for `StandardScaler/MinMaxScaler`), and outliers have less impact on tree-based models like this.
 - Housing prices change over time, and HGBR can't inherently learn time-aware patterns, So I have included time-based features like `year_sold`, `is_sold_in_2021`, `years_diff`

External Data Sources to Improve Accuracy

- **Economic Indicators:** Interest rates, inflation, unemployment data from the Reserve Bank of Australia.
- **Neighborhood Information:** Crime rates, school ratings, walkability scores.
- **Real Estate Trends:** Monthly market reports, buyer sentiment indexes.
- **Infrastructure Projects:** Government databases listing upcoming projects (new train lines, hospitals, etc.).

Charts:





2. Incorporating Temporal Features

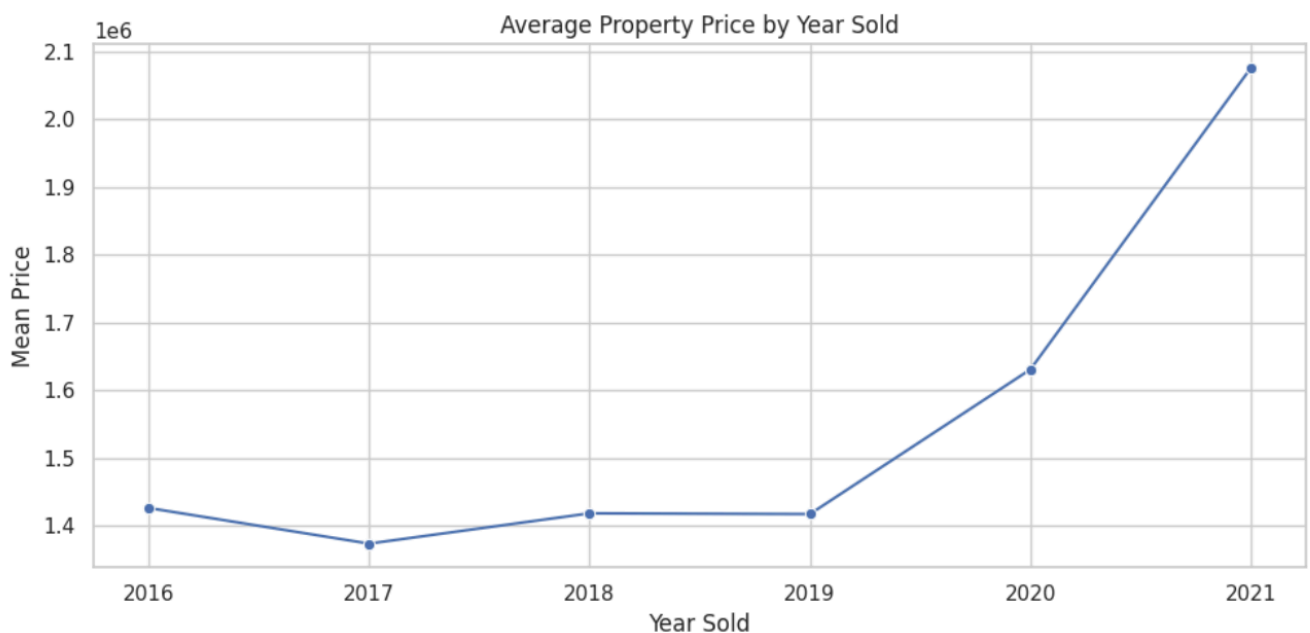
Importance of Temporal Nature in Price Prediction

- **Price Trends:** Real estate prices usually increase over time but can dip due to market crashes or pandemics.
- **Inflation Adjustment:** Nominal prices must be adjusted to reflect real value across years.

Techniques for Incorporating Temporal Features

- **Feature Creation:**
 - year_sold: Extract year from the sale date.
 - is_sold_in_2021: Binary flag for most recent data.
 - years_diff: Difference between sale year and the current year.
- **Inflation Indexing:**
 - Normalize all prices based on an external inflation index to make historical prices comparable. But it didn't work well as we need to predict the price during the time of scale, for predicting the future prices it would be highly desirable.

Temporal Impact Visualization



3. Evaluation Metrics

Regression Task

1. **SVM Regressor:**
 - a. Required normalization of features and removal of outliers.
 - b. Struggled with high-dimensional and non-linearly separable data.
 - c. Performance was sensitive to the kernel and hyperparameters.

- d. **Result:** Underperformed compared to tree-based models, particularly on noisy, skewed data.(MAE ~ 500k)
- 2. **Ridge Regression:**
 - a. Required log transformation and skewness correction to perform adequately.
 - b. Assumes a linear relationship between features and target variable, which did not capture the non-linear patterns in house prices.
 - c. **Result:** Simpler model but lacked predictive power for complex real estate pricing patterns.(MAE ~ 460k)
- 3. **Histogram-Based Gradient Boosting Regressor (HGBRegressor):**
 - o Efficient with missing values and non-linear feature interactions.
 - o Automatically handles outliers and does not require normalization.
 - o Consistently outperformed other models in terms of **Mean Absolute Error (MAE)**.(MAE ~ 306k)

Metrics: Mean Absolute Error

- **Robust to Outliers:** Property prices can vary significantly, from modest homes to multi-million dollar estates. Metrics like MSE (Mean Squared Error) overly penalize large deviations due to squaring, making the model sensitive to price outliers. In contrast, **MAE treats all errors equally**, making it more robust for this use case.

Classification Task

- 1. **K-Nearest Neighbors (KNN):**
 - a. Easy to implement and interpret.
 - b. Performed poorly on rare classes due to class imbalance.
 - c. Computationally expensive for large datasets.
 - d. Sensitive to feature scaling and irrelevant features.
 - e. Result: Struggled with high-dimensional, sparse, and unbalanced categorical data.
- 2. **Random Forest Classifier:**
 - a. Handles imbalanced data well through internal bootstrapping and ensemble voting.
 - b. Robust to overfitting due to averaging across many decision trees.
 - c. Handles non-linearity and interactions among features.
 - d. Does not require normalization or feature scaling.
 - e. Provided the best macro F1-score, indicating balanced performance.

Metrics: F1 Score

- **Class Imbalance:** The dataset includes dominant classes like "House" (4110 records) and rare ones like "Studio" or "Development Site" (as few as 1–5 records). Accuracy would be misleading in this scenario, as a model predicting only the majority class could still score over 80% without learning anything useful about the minority classes. It rewards models that are not only accurate but also capture rare classes without overpredicting them.